

Supplementary Material

Harness of Harness: Iterative Planning, Coding, and Testing for Software Development from Scratch

Contents

A	Method and Implementation Details	2
A.1	HoH Execution Protocol	2
A.2	Role-Specific Prompt Construction	3
A.3	Development Tools and Workspace Operations	5
A.4	Evidence Collection and Representation	6
A.5	Cross-Iteration State Transfer and Evaluation Isolation	9
B	Experimental Protocol	10
B.1	Harness and Model Configurations	10
B.2	Run Configuration and Repetition	10
B.3	Computing Environments	11
B.4	Benchmark Sampling and Evaluated Tasks	11
B.5	Baseline and Budget-Controlled Protocols	14
B.6	Ablation Protocols	15
B.7	Metrics and Resource Accounting	15
B.8	Reproducibility Artifacts	17
C	Complete Experimental Results	18
C.1	GameCraft-Bench Per-Task Scores	18
C.2	FrontierSWE Per-Task Rewards	21
C.3	Budget-Controlled Comparison	23
C.4	Ablation Study	26
C.5	Resource Usage	28
D	Qualitative Analysis	29

Supplementary Overview

This supplementary material provides the implementation and evaluation details that complement the main paper. Section **A** describes the HoH execution protocol, role-specific prompts, development tools, evidence collection, and cross-iteration state transfer. Section **B** specifies the experimental configurations, run controls, computing environments, benchmark subsets, evaluation protocols, and metrics. Section **C** reports complete task-level trajectories, budget-controlled results, ablations, and resource records. Section **D** provides qualitative examples.

A Method and Implementation Details

A.1 HoH Execution Protocol

This section expands the method specification in the main paper into its executable interfaces. Within one experimental condition, the *Project Planner*, *Developer*, and *Quality Assurance (QA) Tester* are three independent invocations of the same harness–model configuration H . Their model and native harness capabilities remain fixed, while role-specific instructions determine what each invocation may read, modify, and return. The three roles coordinate around the same evolving software artifact. The Developer writes to the active project workspace, whereas the Planner consumes materialized documents and the QA Tester inspects an isolated copy of the current artifact. The latter two return structured records rather than modifying the active artifact through an interactive conversation.

Table 1 details the implementation-level inputs and outputs corresponding to the notation in the main paper. The Planner receives the public specification $\mathcal{S}$ and preceding evidence $\mathcal{E}_{t-1}$ and produces the current development document D_t . The Developer receives $\mathcal{S}$ and D_t in the workspace containing A_{t-1} and writes the updated artifact A_t . The QA Tester receives $\mathcal{S}$, D_t , and A_t , executes and inspects the artifact, and produces $\mathcal{E}_t$.

Accordingly, one HoH iteration is implemented by three harness invocations:

$$\begin{aligned} D_t &= \text{Plan}_H(\mathcal{S}, \mathcal{E}_{t-1}), \\ A_t &= \text{Dev}_H(A_{t-1}; \mathcal{S}, D_t), \\ \mathcal{E}_t &= \text{Test}_H(A_t; \mathcal{S}, D_t). \end{aligned} \tag{1}$$

The three invocations share the fixed configuration H , but each receives the role-specific inputs shown in Table 1. The artifact and evidence bundle cross the iteration boundary; within iteration t , D_t provides the common specification for coding and testing.

Table 1 | Inputs, invocation contracts, and materialized outputs of the three HoH roles. All roles use the harness–model configuration associated with the corresponding experimental condition.

Role	Inputs	Invocation contract	Materialized output
Project Planner	$\mathcal{S}$ and $\mathcal{E}_{t-1}$	Select bounded priorities from public requirements and preceding evidence; identify verified functionality to preserve; specify observable acceptance requirements; do not modify production code	Development document D_t
Developer	$\mathcal{S}$, D_t , and the workspace containing A_{t-1}	Address prioritized targets with native coding tools; preserve verified functionality; keep the project buildable and runnable; write changes into the existing workspace	Updated artifact A_t and execution records
QA Tester	$\mathcal{S}$, D_t , A_t , and public execution records	Derive checkable claims; execute and inspect the artifact; associate findings with observable records; distinguish supported functionality from unresolved or insufficiently evidenced requirements	Evidence bundle $\mathcal{E}_t$

A.2 Role-Specific Prompt Construction

Each role prompt is rendered from reusable Markdown modules and runtime values. The fixed modules specify role boundaries, public-information policy, and the output contract; runtime slots insert the public task, current iteration state, materialized documents, and execution records. The public task specification is inserted without modification. At $t = 1$, the Planner’s evidence slot is empty; for $t > 1$, it contains the structured evidence bundle from the preceding artifact. The templates below are schematic, interface-preserving renderings of the runtime prompts: they retain the role contracts and data dependencies used by the method while omitting repeated benchmark-specific examples and checklists. In the templates, `{{runtime_slot}}` denotes substituted content, `/workspace/path` denotes a materialized file or directory, and `[conditional module]` denotes a block included only when its runtime condition is satisfied.

Project Planner Prompt

Template assembly. `[role instruction]` $\oplus$ `[public specification]` $\oplus$ `[preceding evidence]` $\oplus$ `[document scaffold]` $\oplus$ `[output contract]`

Method correspondence. $(\mathcal{S}, \mathcal{E}_{t-1}) \xrightarrow{\text{Plan}_H} D_t$

`/role/project-planner`

You are the Project Planner for iteration `{{loop_index}}` of an iterative software-development run. This is a planning-only harness invocation. Do not implement, edit, test, or inspect production code. Return only a prioritization overlay for the deterministic development document.

`/source-of-truth`

The public `{{task_source_name}}` specification below is the complete product source of truth. Do not use benchmark scores, hidden tests, private rubrics, evaluator feedback, or other non-public information.

`{{public_task_instruction}}`

`/previous-iteration-evidence`

`{{evidence_packet}}`

For iteration 1, there is no previous-iteration evidence. For later iterations, identify verified functionality to preserve, visible bugs and unmet requirements to repair, and evidence that remains insufficient. Do not request or reconstruct the previous development document.

`/planning-policy`

Prioritize blockers and regressions before product extensions. Select at most three achievable priorities already supported by the document scaffold. Convert each priority into a concrete implementation target and an observable validation requirement; avoid broad rewrites or unrelated architecture changes.

`/document-scaffold`

`{{scaffold_document}}`

`/output-contract`

Return only the following Markdown structure:

Project Planner Priorities

Priority Order

1. **Priority name** – action and observable outcome

Preservation Gate

– Working functionality and evidence that must not regress

Acceptance Gate

– Smallest end-to-end validation for the selected priorities

Developer Prompt

Template assembly. [benchmark guidance] $\oplus$ [iteration wrapper] $\oplus$ [warm-start block] $\oplus$ [development document]

Method correspondence. $(A_{t-1}; S, D_t) \xrightarrow{\text{Dev}_H} A_t$

/role/developer

You are the Developer for iteration {{loop_index}}. Build or improve the complete project at /workspace/game. Treat the public task instruction as the PRD and the current development document as the implementation and validation brief for this iteration.

/iteration-context

Outer-loop attempt {{attempt}}

[if warm-started: {{warm_start_section}}]

Continue from the artifact already present in /workspace/game. Preserve verified functionality and repair the next observable gap rather than replacing a working project with a smaller reset.

/development-document

Current document: {{development_doc_filename}}

Planning inputs: {{planning_inputs}}

Build status: {{build_ok}}

Observed demos: {{num_demos}}

{{public_task_instruction}}

{{focus_items}}

{{preserve_visible_section}}

{{development_brief}}

{{evidence_history_section}}

/development-policy

Repair build and runtime blockers first, then address the ordered targets in the document. Use the harness's native file, repository, shell, build, execution, and local-testing tools. Keep the project launchable and make every claimed mechanic observable through a valid replay trace. Public assets may be read from /workspace/assets/library and /workspace/assets/library-oga; copied assets count only when they are visibly used by the artifact.

/output-contract

Leave the updated artifact in /workspace/game, including /workspace/game/project.godot, a launchable main scene, and valid /workspace/game/demo_outputs/*.json traces. Preserve the public runtime records required for QA testing.

QA Tester Prompt

Template assembly. [tester role] $\oplus$ [phase instruction] $\oplus$ [development document] $\oplus$ [media manifest] $\oplus$ [phase-specific output contract]

Method correspondence. $(A_t; S, D_t) \xrightarrow{\text{Test}_H} \mathcal{E}_t$

/role/qa-tester

You are the QA Tester for iteration {{loop_index}}. Review the updated artifact as a player-facing product using only the public task text, the current development document, visible project files, screenshots, replay traces, and videos. Do not modify production code.

{{tester_phase_instruction}}

/inputs

Task directory: {{task_dir}}

Trial: {{trial_name}}

Trial directory: {{trial_dir}}

QA Tester Prompt (continued)

```
Development document: {{development_doc_filename}}  
  
/development-document  
{{development_document}}  
  
/public-visual-evidence  
{{media_lines}}  
  
/assessment-policy  
Derive checkable claims from the public requirements and validation targets. Assign a claim–evidence  
record to the verified subset only when the cited execution records provide sufficient observable support.  
Record visible failures, regressions, unmet requirements, and insufficient evidence as gaps. Inspect replay  
event types and reject traces that cannot be reproduced through the public mouse, keyboard, and  
wait-event interface.  
  
/restrictions  
Do not read, list, infer from, or summarize /tests, benchmark scores, private evaluator files, hidden task  
metadata, formulas, or host-only materials outside the public task and generated artifact.  
  
/output-contract  
Write visual_playtest_report.md and visual_playtest_report.json. Each report records status, cited  
evidence, player impact, issue ownership, and a concrete recommendation.  
[next-loop phase: remaining bugs, preservation records, and next-loop goals]  
{{tester_phase_output_contract}}
```

The QA prompt requests evidence-bearing findings in a benchmark-appropriate JSON report. The report need not reproduce the mathematical tuple notation verbatim. After the QA invocation, the benchmark adapter normalizes its claims, cited execution records, and statuses into the evidence bundle $\mathcal{E}_t$ used in the main paper. Thus, Test_H denotes the complete testing interface, including both evidence collection by the QA Tester and deterministic normalization of its report.

A.3 Development Tools and Workspace Operations

HoH does not replace the tools exposed by the underlying coding harness. Instead, the Developer uses those tools under the current development document and writes all changes to the same project workspace. The workspace contains source code, configuration, project resources, and any public runtime artifacts produced during development. Warm-starting therefore preserves not only source files but also the project structure and resources required to continue development from A_{t-1} .

Table 2 summarizes the principal capabilities used by the implementation. The exact commands depend on the selected harness and benchmark, but the role of each capability is fixed across iterations.

Table 2 | Development and inspection capabilities used by HoH. Private benchmark evaluators and their outputs are excluded from these interfaces.

Capability	Representative operations	Retained records
Project operations	Inspect and edit source files, configuration, assets, and repository state	Updated files and change state
Build and execution	Invoke shell commands, build the project, launch the artifact, and run public local checks	Exit status, standard output, standard error, and runtime logs
Game interaction	Launch Godot scenes, exercise controls, and execute deterministic interaction traces	Replay traces and observable state transitions
Visual inspection	Capture screenshots or videos and inspect visible asset use, interface state, feedback, and result screens	Media manifest and referenced frames
Task-specific tools	Use public benchmark containers, dependencies, and task-provided verification utilities where available	Public test and execution results

For GameCraft-Bench, A_t is a Godot project containing source scripts, scenes, configuration, assets, and replay outputs. The benchmark adapter materializes the current development document as additional context for the Developer and records the harness command, process outcome, and resulting trial. For FrontierSWE, the same interfaces operate on the task repository and its public execution environment. Benchmark-specific adapters change how an artifact is launched and observed; they do not change the planning, coding, or testing roles.

A.4 Evidence Collection and Representation

The QA stage and benchmark adapter together convert observable behavior into the claim–evidence records defined in the main paper. The correspondence is

$$\begin{aligned}
 C_t &= \text{Claims}(\mathcal{S}, D_t), \\
 r_i &= \text{Observe}(A_t, c_i), \\
 s_i &= \text{Assess}(c_i, r_i), \\
 \mathcal{E}_t &= \{(c_i, r_i, s_i)\}_{c_i \in C_t}.
 \end{aligned} \tag{2}$$

Here, C_t contains the checkable claims, r_i denotes the public execution records collected for claim c_i , and s_i is the normalized QA status. Claims are instantiated from the public requirements, current development targets, preservation constraints, and validation requirements.

Evidence collection first executes or inspects the artifact using the capabilities above. Build and test outcomes establish whether the artifact can run; runtime logs and traces expose state transitions; screenshots, videos, and replays provide player-visible observations; and asset inspection determines whether project resources are used in the executed artifact. Source-code presence alone is not treated as behavioral verification.

The QA Tester then assesses every claim against its cited records. A record is placed in $\mathcal{E}_t^{\text{ver}}$ only when the evidence visibly supports the corresponding claim. Observed failures, unmet requirements, regression risks, and claims without sufficient evidence are placed in $\mathcal{E}_t^{\text{gap}}$. Formally, the two subsets

are

$$\begin{aligned}\mathcal{E}_t^{\text{ver}} &= \{(c_i, r_i, s_i) \in \mathcal{E}_t \mid s_i = \text{verified}\}, \\ \mathcal{E}_t^{\text{gap}} &= \{(c_i, r_i, s_i) \in \mathcal{E}_t \mid s_i = \text{gap}\}.\end{aligned}\tag{3}$$

They form a disjoint partition of the evidence bundle:

$$\mathcal{E}_t = \mathcal{E}_t^{\text{ver}} \cup \mathcal{E}_t^{\text{gap}}, \quad \mathcal{E}_t^{\text{ver}} \cap \mathcal{E}_t^{\text{gap}} = \emptyset.\tag{4}$$

The implementation retains both a human-readable tester report and structured records for subsequent planning. Listing 1 shows a normalized excerpt organized according to the verified- and gap-record subsets above. Each record preserves the claim, the public execution records used to assess it, and the resulting status. The final block shows how these records are converted into planning inputs for the next iteration.

Listing 1 Normalized structured evidence report. Each claim is linked to its public execution records and QA status. Verified records yield preservation constraints, whereas gap records yield update targets and follow-up validation requirements for the next iteration.

```

1  {
2    "iteration": 2,
3    "qa_status": "partial",
4    "verified_records": [
5      {
6        "claim_id": "player_control",
7        "claim": "Player input changes avatar motion.",
8        "execution_records": [
9          {
10             "type": "replay",
11             "path": "replays/core_loop.json",
12             "observation": "Left and right inputs move the avatar."
13           },
14           {
15             "type": "runtime_trace",
16             "path": "traces/core_loop.json",
17             "observation": "Position changes after each input event."
18           }
19         ],
20         "status": "verified"
21       }
22     ],
23     "gap_records": [
24       {
25         "claim_id": "result_state",
26         "claim": "Completing the objective produces a visible result.",
27         "execution_records": [
28           {
29             "type": "screenshot",
30             "path": "screenshots/frame_018.png",
31             "observation": "The objective ends without a result screen."
32           }
33         ],
34         "status": "gap",
35         "player_impact": "Completion is not visible to the player.",
36         "recommended_update": "Add and replay a result state."
37       }
38     ],
39     "planner_handoff": {
40       "preservation_constraints": [
41         "Preserve verified player movement."
42       ],
43       "update_targets": [
44         "Implement a visible completion state."
45       ],
46       "validation_requirements": [
47         "Replay objective completion through the result screen."
48       ]
49     }
50 }

```

For GameCraft-Bench, the evidence bundle is materialized through a screenshot manifest, playtest report, structured status record, replay traces, and tester logs. FrontierSWE uses the same claim-evidence abstraction with the public execution and task-specific test records available in its repository

environment.

A.5 Cross-Iteration State Transfer and Evaluation Isolation

The implementation preserves the two state channels defined in the main paper. The artifact channel carries the updated project A_t into the next Developer invocation, while the evidence channel carries $\mathcal{E}_t$ into the next Planner invocation. Within iteration t , D_t is the shared specification for coding and testing; the next Planner constructs a new document from $\mathcal{S}$ and $\mathcal{E}_t$ rather than treating D_t as a third persistent state channel.

The implementation retains the development document, harness command, process logs, public media manifest, structured QA report, and resulting project workspace for each iteration. These records make the transition from A_{t-1} to A_t and the construction of $\mathcal{E}_t$ auditable without exposing evaluator-only information.

Benchmark evaluation is separated from development. Hidden tests, benchmark scores, private rubrics, evaluation formulas, and evaluator rationales are not included in the role prompts or evidence bundle and are never returned to a subsequent iteration. HoH therefore adapts to observable execution and QA findings while the public task specification remains the authoritative requirement source.

B Experimental Protocol

B.1 Harness and Model Configurations

Table 3 lists the three configurations used throughout the main experiments. Harness versions, models, and exposed reasoning settings are held fixed between Vanilla and HoH within each configuration. The main HoH results use $T = 3$.

Table 3 | Harness–model configurations used in the experiments.

Harness	Version	Model	Reasoning setting
Codex CLI ¹	0.142.5	GPT-5.5	High
OpenCode ²	1.14.30	DeepSeek-V4-Pro	—
Pi Coding Agent ³	0.80.10	MiniMax-M3	Client-side high

B.2 Run Configuration and Repetition

The main experiments use three HoH iterations. Vanilla performs one standard development pass, and the budget-controlled experiment additionally evaluates two and three sequential Vanilla development passes. HoH reports the artifact produced after the prescribed iteration budget; intermediate artifacts are evaluated only for analysis and are not selected using benchmark scores. Evaluator outputs are not returned to the planning, coding, or testing stages.

Table 4 summarizes the run structure used for each experiment. Every reported task–condition score is obtained from one valid run. When an attempt fails because of an infrastructure or model-provider transport error, the failed attempt is replaced rather than included as an additional replicate. Aggregate scores therefore average over tasks, not over multiple generations of the same task–condition pair.

Table 4 | Run configurations used in the reported experiments.

Experiment	Benchmark	Development structure	Runs per task–condition
Main comparison	GameCraft-Bench and FrontierSWE	Vanilla: one coding pass; HoH: $T = 3$ iterations	1
Budget comparison	GameCraft-Bench	Vanilla Continuation: one, two, or three coding passes; HoH: $T = 3$	1
Ablation study	GameCraft-Bench	Full HoH and each ablation: $T = 3$	1

Task–condition runs start from separate copies of the benchmark-provided workspace. Within a harness–model configuration, Vanilla and HoH use the same model, native harness settings, public task materials, and benchmark tools. The selected clients do not expose a common reproducible

¹<https://github.com/openai/codex>

²<https://github.com/anomalyco/opencode>

³<https://github.com/earendil-works/pi>

generation seed, and we do not override temperature or top- p ; the corresponding client and provider defaults are used throughout. The fixed seeds reported below control task sampling and statistical resampling rather than model generation.

B.3 Computing Environments

The two benchmarks use different execution environments because they exercise different software artifacts. Table 5 records the shared runtime components. Model inference is provided through the remote services associated with the configurations in Table 3; the listed machines execute the harnesses, generated artifacts, and benchmark verifiers.

Table 5 | Computing environments used for artifact development and evaluation. FrontierSWE task images retain their benchmark-defined dependencies and resource declarations.

Component	GameCraft-Bench	FrontierSWE
Isolation	Run-local workspace in a local-subprocess environment	Official task container launched through a run-local Docker daemon
Host	Ubuntu 24.04.3; Intel Core i7-14700; 64 GB RAM	Linux compute workers; NVIDIA H200 for tasks requiring a GPU
Runtime	Python 3.12.3; Godot 4.6.2; Xvfb-backed display capture	Docker with the official task-specific software image
Resource control	No task-specific GPU allocation	CPU, memory, storage, and GPU limits specified by each official task

FrontierSWE uses a Docker-in-Docker execution design. An outer execution container starts a run-local Docker daemon, which launches the official task-specific image. The inner container receives the CPU, memory, storage, and accelerator limits declared by that task. GPU passthrough is enabled only for tasks that request an accelerator. This preserves the task software stack and prevents dependencies from one task from affecting another.

B.4 Benchmark Sampling and Evaluated Tasks

Table 6 summarizes the benchmark subsets used in the main experiments. GameCraft-Bench [?] is sampled by its 15 public game families and is additionally organized into five coarse reporting groups for analysis. FrontierSWE [?] uses the benchmark’s three official categories.

Table 6 | Composition of the evaluated benchmark subsets. Counts refer to the tasks used for every harness–model configuration in the main experiments.

Benchmark	Tasks	Fine categories	Reporting groups	Tasks per category
GameCraft-Bench	45	15	5	3 per family
FrontierSWE	15	3	3	4 / 9 / 2

B.4.1 GameCraft-Bench

We use a fixed 45-task subset with three tasks from each of the benchmark’s 15 public game families. The subset was constructed incrementally from a fixed earlier subset and completed to three tasks

per family by seeded stratified sampling (seed 20260707), without reference to model scores.

For compact reporting, we group the 15 families into five coarse categories, each containing three families and nine tasks: Action contains Platformer, Shooter, and Roguelike; Timing contains Racing, Rhythm, and Sports; Strategy contains Strategy, Card Game, and Puzzle; Simulation contains Tycoon, Idle, and Simulation; and Adventure contains Horror, Open World, and Visual Novel. These five groups are introduced only for aggregate analysis; all task scores continue to use the benchmark's original family definitions.

Table 7 | GameCraft-Bench reporting groups, benchmark families, and sampled tasks. Each family contributes three tasks.

Family	Task	Benchmark identifier
Action		
Platformer	Momentum Lab	platformer-momentum-lab
	Ivory Beats	platformer-ivory-beats
	Thunder Valkyrie	platformer-thunder-valkyrie
Shooter	Void Patrol	shooter-void-patrol
	Wave Commander	shooter-wave-commander
	Hotline Heist	shooter-hotline-heist
Roguelike	Dungeon Shop	roguelike-dungeon-shop
	Breach Tactics	roguelike-breach-tactics
	Void Harvest	roguelike-action-void-harvest
Timing		
Racing	Drift Circuit	racing-drift-circuit
	Rocket Trials	racing-rocket-trials
	Trick Runner	racing-trick-runner
Rhythm	Note Highway	rhythm-note-highway
	Beat Dungeon	rhythm-beat-dungeon
	Garden	rhythm-garden
Sports	Skateboard Park	sports-skateboard-park
	Boxing Gym	sports-boxing-gym
	Archery Quest	sports-archery-quest
Strategy		
Strategy	Tower Defense	strategy-towerdefense
	Chess Variant	strategy-chess-variant
	Spell Tactics	strategy-spell-tactics
Card Game	Spire Descent	cardgame-spire-descent
	Poker Roguelike	cardgame-poker-roguelike
	Autobattler	cardgame-autobattler
Puzzle	Sokoban Dungeon	puzzle-sokoban-dungeon
	Circuit Wizard	puzzle-circuit-wizard
	Pipe Crisis	puzzle-pipe-crisis
Simulation		
Tycoon	Space Colony	tycoon-space-colony
	Pirate Port	tycoon-pirate-port
	Wildhaven	tycoon-wildhaven
Idle	Ant Empire	idle-ant-empire
	Factory Planet	idle-factory-planet
	Dungeon Guild	idle-dungeon-guild

Table 7 (continued)

Family	Task	Benchmark identifier
Simulation	Kitchen Rush	simulation-kitchen-rush
	Air Control	simulation-air-control
	Border Check	simulation-border-check
Adventure		
Horror	Floor 13	horror-floor-13
	Dollhouse	horror-dollhouse
	Lighthouse	horror-lighthouse
Open World	Sky Islands	openworld-sky-islands
	Airship Trader	openworld-airship-trader
	Bounty	openworld-bounty
Visual Novel	Detective Noir	visualnovel-detective-noir
	Arcane Academy	visualnovel-arcaneacademy
	Time Paradox	visualnovel-time-paradox

B.4.2 FrontierSWE

We evaluate 15 FrontierSWE tasks under the benchmark’s official taxonomy: four Implementation tasks, nine Performance tasks, and two Research tasks. We additionally distinguish tasks that construct an independent deliverable from a scaffold or task specification from those that optimize an existing system. Under this criterion, 10 tasks are labeled end-to-end and five are labeled optimization.

Table 8 | FrontierSWE tasks grouped by official category and construction scope.

Task	Scope	Brief task description
Implementation		
Dart Style Haskell	End-to-end	Reimplement the Dart formatter in Haskell as a Cabal-built executable compatible with the relevant CLI behavior and golden formatting cases.
Git to Zig	End-to-end	Reimplement Git 2.47 as a Zig binary compatible with Git’s CLI, output, and exit-code behavior, without reusing the existing Git implementation or network access.
Lua Native Compiler	End-to-end	Compile Lua 5.4 bytecode to a standalone native x86-64 executable with reference-equivalent output, rather than an interpreter or API wrapper.
PostgreSQL–SQLite Wire Adapter	End-to-end	Build a Zig server backed by SQLite that emulates the required PostgreSQL server, wire-protocol, lifecycle, and CLI behavior.
Performance		
Cranelflift Codegen Optimization	Optimization	Optimize compiled WebAssembly runtime performance in Wasmtime’s Cranelflift backend, subject to correctness gates and weighted speedup scoring.
Dependent Type Checker	End-to-end	Implement a correct, high-throughput Martin-Löf type checker in Rust; correctness thresholds must be met before throughput is scored.
FFmpeg Swscale Rewrite	End-to-end	Rewrite libswscale in Zig or Rust behind its required C ABI, with image-quality gates before geometric-mean speedup scoring.

Table 8 (continued)

Task	Scope	Brief task description
Granite Mamba2 Inference Optimization	Optimization	Optimize a standalone Granite Mamba2 layer while preserving CUDA bfloat16 outputs and cache behavior across the evaluated workloads.
Inference System Optimization	Optimization	Accelerate a Qwen-based SGLang serving system while preserving token-level output equivalence under latency and throughput workloads.
Libexpat to x86 Assembly	End-to-end	Reimplement the required libexpat API as an independent x86-64 assembly shared library without delegating to the existing implementation.
Notebook Compression	End-to-end	Build a lossless domain-specific notebook compressor with fit, compress, and decompress interfaces; exact recovery is required before compression ratio is scored.
Pyright Type-Checking Optimization	Optimization	Optimize Pyright’s type-evaluation hot paths while preserving build success, all required tests, and reference-equivalent diagnostics.
Revideo Performance Optimization	Optimization	Optimize Revideo’s programmatic rendering pipeline without frame skipping, quality reduction, resolution changes, or visible-output deviations.
Research		
Optimizer Design	End-to-end	Implement one torch.optim.Optimizer and a shared hyperparameter configuration that generalizes across heterogeneous ML workloads.
PCQM4Mv2 Autoresearch	End-to-end	Train a 2D molecular-graph regressor under data, model, and parameter constraints to minimize the evaluated molecular-property error.

Two official FrontierSWE tasks are not included in the evaluated subset. Their omission is determined by execution requirements rather than model outcomes.

Table 9 | FrontierSWE tasks excluded from the evaluated subset.

Task	Reason
frogsgame-rl	Requires authenticated access to the external Tinker API, which was unavailable in the evaluation environment.
modular-stack-wan21	Requires a Modular MAX software stack needing an NVIDIA driver of at least 580 (CUDA 13), whereas the available H200 worker used driver 570.133.20.

B.5 Baseline and Budget-Controlled Protocols

Vanilla. Vanilla uses the corresponding harness–model configuration without the HoH protocol and performs one standard development pass from the benchmark-provided initial artifact.

Vanilla Continuation. The budget-controlled comparison extends the selected Vanilla artifact through two additional invocations of the same harness–model configuration. Let A_1^{VC} denote the

artifact produced by the standard Vanilla pass. For $k \in \{2, 3\}$, Vanilla Continuation applies

$$A_k^{\text{VC}} = \text{Dev}_H \left(A_{k-1}^{\text{VC}}; \mathcal{S}, p_{\text{cont}} \right), \quad (5)$$

where p_{cont} is the fixed instruction shown below. Thus, each additional pass starts from the latest artifact, but receives neither a development document nor evidence from a separate QA Tester invocation.

Vanilla Continuation Prompt

Continue developing and testing the current game.

The comparison therefore holds the initial task set and harness–model configuration fixed while separating repeated coding passes from the planning–coding–testing structure of HoH.

B.6 Ablation Protocols

The ablations retain the three-iteration budget and the same harness–model configuration as full HoH. Relative to the full iteration in Eq. 1, each variant changes one cross-iteration input while leaving the remaining interfaces unchanged:

$$\begin{aligned} \text{w/o Plan Update: } D_t &= D_1, & t > 1, \\ \text{w/o Evidence Feedback: } D_t &= \text{Plan}_H(\mathcal{S}, \emptyset), \\ \text{w/o Warm-Start: } A_t &= \text{Dev}_H(A_0; \mathcal{S}, D_t). \end{aligned} \quad (6)$$

In *w/o Plan Update*, the first development document is reused in all later iterations, although coding and testing continue on the evolving artifact. In *w/o Evidence Feedback*, the QA Tester still evaluates each updated artifact, but its evidence is withheld from the next Planner invocation. In *w/o Warm-Start*, evidence-conditioned planning and QA testing remain active, while every Developer invocation begins from the benchmark-provided initial artifact A_0 .

Table 10 | Information channels retained by the ablation variants.

Variant	Development document	Coding start	Evidence for next plan
Full HoH	Updated from preceding evidence	A_{t-1}	$\mathcal{E}_{t-1}$
w/o Plan Update	Fixed to D_1	A_{t-1}	Not consumed
w/o Evidence Feedback	Updated from $\mathcal{S}$ only	A_{t-1}	Withheld
w/o Warm-Start	Updated from preceding evidence	A_0	$\mathcal{E}_{t-1}$

B.7 Metrics and Resource Accounting

GameCraft-Bench dimensions. GameCraft-Bench evaluates runnable game artifacts along four dimensions. *Core Mechanics* measures implementation of the required gameplay mechanics and interaction loop. *Content Depth* measures the breadth and variety of stages, challenges, objectives, and progression. *Functional Visuals* measures the visibility, readability, and feedback of gameplay states. *Art and Presentation* measures visual coherence, asset quality, interface styling, and polish. Let M , D , V , and A denote the mean rubric-item scores for these four dimensions, respectively.

GameCraft-Bench Overall. The benchmark combines the four dimensions as

$$\text{Overall} = 100B(0.15M + 0.35D + 0.15V + 0.35A), \quad (7)$$

where $B = 1$ if the game artifact compiles and runs and $B = 0$ otherwise.

FrontierSWE reward. We report the task-specific official reward and its mean over the 15 evaluated tasks.

Task-level aggregation. For a benchmark task set $\mathcal{B}$ and evaluated condition c , the reported aggregate is the unweighted mean of its task-level scores:

$$\bar{s}_{\mathcal{B}}(c) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} s_i(c). \quad (8)$$

For GameCraft-Bench, s_i is the 0–100 Overall score and $|\mathcal{B}| = 45$; for FrontierSWE, s_i is the official reward and $|\mathcal{B}| = 15$.

Bootstrap uncertainty. The 95% confidence intervals for the GameCraft-Bench component analysis in the main paper are percentile intervals from 20,000 task-bootstrap resamples. Tasks are sampled with replacement, all four component scores for a sampled task are retained together, and means are recomputed for every resample. The bootstrap uses seed 20260729.

Model-interaction volume. Token counts include the provider-reported input and output tokens from coding-harness model calls and exclude benchmark evaluation. Input totals may include cached context reads, whose accounting differs across providers. Let $\mathcal{I}_i(c)$ denote the model calls made for task i under condition c . Cumulative token use, in millions of tokens, is

$$C_i(c) = 10^{-6} \sum_{j \in \mathcal{I}_i(c)} (n_j^{\text{in}} + n_j^{\text{out}}), \quad \bar{C}(c) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} C_i(c). \quad (9)$$

We compare token usage within each harness–model configuration because cache accounting differs across providers. In the budget-controlled comparison, we also report the quality gained per additional million tokens relative to Vanilla:

$$\eta(c) = \frac{\bar{s}_{\text{GC}}(c) - \bar{s}_{\text{GC}}(\text{Vanilla})}{\bar{C}(c) - \bar{C}(\text{Vanilla})}. \quad (10)$$

Official dominance. We apply the official dominance procedure to the final 15-task subset. Within each domain, the comparison pool contains all $3 \times 4 = 12$ system-condition configurations: the three systems under Vanilla, HoH@1, HoH@2, and HoH@3. For a configuration a , let d denote a domain, let t denote a task in that domain, and let $r_{a,t}$ denote a 's official reward on t .

All pairwise comparisons occur on the same task. For any opponent j among the other 11 configurations, the comparison score is

$$s(x, y) = \begin{cases} 1, & x > y, \\ 0.5, & x = y, \\ 0, & x < y. \end{cases} \quad (11)$$

The task-level dominance of a is therefore

$$\text{Dominance}_{d,t}(a) = \frac{1}{11} \sum_{j \neq a} s(r_{a,t}, r_{j,t}). \quad (12)$$

Equivalently, this quantity is the expected comparison score when the opponent is selected uniformly from the other 11 configurations. The implementation computes this expectation exactly by averaging over all 11 opponents. The denominator is 11 because a is compared with every other member of the 12-configuration pool, but not with itself. Domain-level dominance averages these values equally over the N_d tasks in domain d :

$$\text{Dominance}_d(a) = \frac{1}{N_d} \sum_{t=1}^{N_d} \left[\frac{1}{11} \sum_{j \neq a} s(r_{a,t}, r_{j,t}) \right]. \quad (13)$$

Here, N_d is 4, 9, and 2 for Implementation, Performance, and Research, respectively. The reported FrontierSWE dominance is the macro average over the three domains:

$$\text{Dominance}(a) = \frac{1}{3} \sum_d \text{Dominance}_d(a). \quad (14)$$

In this expression, d ranges over Implementation, Performance, and Research, and each domain receives equal weight.

B.8 Reproducibility Artifacts

The anonymous code package accompanying the submission contains the core HoH implementation, role prompt templates, the GameCraft-Bench adapter, and the necessary wrappers for the evaluated harness-model configurations. Benchmark repositories, task data, raw run artifacts, analysis records, environment files, private credentials, provider secrets, and benchmark-hidden evaluator contents are not included.

C Complete Experimental Results

C.1 GameCraft-Bench Per-Task Scores

Figure 1 summarizes Vanilla and HoH@3 over the five reporting groups before the complete task-level results. Each bar is the unweighted mean of the nine tasks in that group.

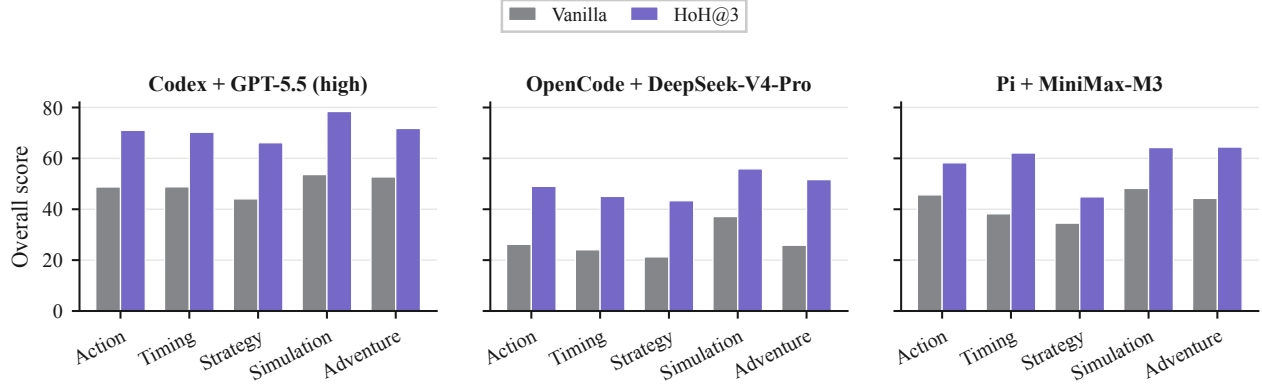


Figure 1 | GameCraft-Bench Overall scores by reporting group and harness-model configuration. Each group contains nine tasks.

Tables 11–13 report the four observed conditions for every sampled GameCraft-Bench task. Scores are converted to the benchmark’s 0–100 presentation scale and grouped using the five coarse categories defined in Table 7. The final column reports the task-specific change from Vanilla to HoH@3.

Table 11 | Complete GameCraft-Bench task scores for Codex + GPT-5.5 (high). Scores use the benchmark’s 0–100 scale; Δ denotes HoH@3 minus Vanilla.

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Action						
Platformer	Momentum Lab	34.05	64.44	56.15	70.61	+36.56
	Ivory Beats	46.81	73.28	78.55	85.42	+38.61
	Thunder Valkyrie	53.51	68.24	71.82	73.67	+20.15
Shooter	Void Patrol	58.59	73.90	76.02	87.83	+29.23
	Wave Commander	66.97	69.35	74.64	75.25	+8.28
	Hotline Heist	43.37	35.71	59.17	62.81	+19.44
Roguelike	Dungeon Shop	40.89	50.55	50.15	65.09	+24.20
	Breach Tactics	53.44	55.67	57.11	61.09	+7.65
	Void Harvest	41.05	46.39	55.43	57.43	+16.38
Timing						
Racing	Drift Circuit	43.58	58.29	60.29	70.08	+26.50
	Rocket Trials	45.19	61.32	68.00	70.31	+25.12
	Trick Runner	51.50	36.09	54.44	64.78	+13.28
Rhythm	Note Highway	39.44	31.35	56.15	64.68	+25.24
	Beat Dungeon	47.94	56.95	60.55	60.81	+12.88
	Garden	52.75	65.33	69.01	70.69	+17.94
Sports	Skateboard Park	57.64	69.51	72.62	81.14	+23.49
	Boxing Gym	43.08	40.34	45.61	73.14	+30.06
	Archery Quest	58.09	64.96	71.56	76.69	+18.60

Table 11 (continued)

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Strategy						
Strategy	Tower Defense	58.85	63.98	64.12	76.92	+18.07
	Chess Variant	30.04	56.93	52.83	59.72	+29.68
	Spell Tactics	53.10	47.39	60.70	63.32	+10.22
Card Game	Spire Descent	25.85	45.47	54.81	61.92	+36.07
	Poker Roguelike	44.66	63.94	67.99	69.27	+24.61
	Autobattler	55.06	72.39	71.28	72.22	+17.16
Puzzle	Sokoban Dungeon	55.75	58.98	58.56	70.13	+14.38
	Circuit Wizard	31.35	38.97	40.82	49.52	+18.17
	Pipe Crisis	41.88	65.07	68.65	72.17	+30.28
Simulation						
Tycoon	Space Colony	36.88	64.60	69.15	76.55	+39.67
	Pirate Port	51.32	63.06	70.75	77.81	+26.48
	Wildhaven	49.43	74.09	77.78	80.39	+30.96
Idle	Ant Empire	65.52	71.42	69.64	87.88	+22.36
	Factory Planet	75.16	78.94	82.27	87.78	+12.63
	Dungeon Guild	63.92	68.94	76.95	79.50	+15.57
Simulation	Kitchen Rush	42.62	48.07	65.64	73.38	+30.75
	Air Control	54.24	63.96	66.72	69.73	+15.50
	Border Check	43.58	67.70	67.00	72.74	+29.17
Adventure						
Horror	Floor 13	48.68	70.73	68.56	73.41	+24.73
	Dollhouse	56.33	64.28	66.93	72.59	+16.27
	Lighthouse	51.95	63.89	71.47	80.90	+28.95
Open World	Sky Islands	53.25	47.19	60.91	68.25	+15.00
	Airship Trader	59.58	73.60	74.40	74.41	+14.83
	Bounty	47.00	67.47	63.31	68.27	+21.27
Visual Novel	Detective Noir	47.46	63.15	55.54	65.31	+17.85
	Arcane Academy	59.30	37.60	62.71	70.69	+11.39
	Time Paradox	50.63	63.28	71.15	71.97	+21.34
Mean		49.58	59.71	64.84	71.52	+21.93

Table 12 | Complete GameCraft-Bench task scores for OpenCode + DeepSeek-V4-Pro. Scores use the benchmark’s 0–100 scale; Δ denotes HoH@3 minus Vanilla.

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Action						
Platformer	Momentum Lab	14.33	23.13	32.92	30.75	+16.42
	Ivory Beats	31.73	45.78	42.09	61.08	+29.35
	Thunder Valkyrie	19.37	28.65	52.20	60.54	+41.17
Shooter	Void Patrol	37.12	57.44	53.49	57.77	+20.66
	Wave Commander	14.74	2.31	35.61	46.46	+31.71
	Hotline Heist	33.85	12.78	33.42	38.05	+4.20
Roguelike	Dungeon Shop	39.98	41.52	53.22	46.50	+6.52
	Breach Tactics	35.28	23.77	36.76	46.99	+11.71
	Void Harvest	9.50	14.34	49.29	52.83	+43.33
Timing						
Racing	Drift Circuit	37.87	31.48	39.17	38.69	+0.82
	Rocket Trials	2.41	3.02	19.90	22.68	+20.27

Table 12 (continued)

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Rhythm	Trick Runner	14.37	19.62	25.26	39.24	+24.86
	Note Highway	25.25	20.88	29.61	40.34	+15.09
	Beat Dungeon	3.06	18.60	17.05	23.81	+20.74
	Garden	49.29	58.11	55.21	63.74	+14.44
Sports	Skateboard Park	19.28	24.69	46.28	57.60	+38.32
	Boxing Gym	31.31	40.69	42.25	55.75	+24.44
	Archery Quest	33.57	29.50	54.31	63.62	+30.06
Strategy						
Strategy	Tower Defense	29.47	25.89	39.18	49.54	+20.07
	Chess Variant	18.76	23.69	40.45	38.51	+19.75
	Spell Tactics	15.03	35.17	21.57	42.05	+27.02
Card Game	Spire Descent	13.33	4.38	19.47	23.00	+9.67
	Poker Roguelike	20.50	24.89	38.77	57.90	+37.40
	Autobattler	30.76	18.75	23.12	18.65	-12.12
Puzzle	Sokoban Dungeon	34.27	33.67	51.95	51.67	+17.40
	Circuit Wizard	3.90	14.56	3.90	35.84	+31.95
	Pipe Crisis	25.39	14.55	63.17	72.90	+47.50
Simulation						
Tycoon	Space Colony	34.24	47.98	57.87	60.27	+26.03
	Pirate Port	25.89	61.33	52.00	52.16	+26.26
	Wildhaven	47.66	46.52	55.13	66.07	+18.41
Idle	Ant Empire	40.73	42.64	55.49	49.08	+8.36
	Factory Planet	38.17	35.20	41.02	67.84	+29.66
	Dungeon Guild	15.66	32.89	69.71	61.34	+45.67
Simulation	Kitchen Rush	33.31	33.41	36.44	39.96	+6.64
	Air Control	28.23	35.40	33.49	35.30	+7.07
	Border Check	70.22	58.34	69.22	70.74	+0.52
Adventure						
Horror	Floor 13	29.22	14.56	41.53	42.12	+12.90
	Dollhouse	8.75	10.95	13.00	54.39	+45.64
	Lighthouse	52.11	35.78	46.30	83.33	+31.23
Open World	Sky Islands	24.37	11.87	28.69	46.51	+22.15
	Airship Trader	38.94	35.41	54.64	62.37	+23.42
	Bounty	7.09	13.66	0.73	25.35	+18.26
Visual Novel	Detective Noir	22.22	18.06	56.59	61.57	+39.35
	Arcane Academy	28.76	26.72	51.36	49.05	+20.29
	Time Paradox	21.13	34.96	31.62	40.04	+18.91
Mean		26.90	28.61	40.32	48.98	+22.08

Table 13 | Complete GameCraft-Bench task scores for Pi + MiniMax-M3. Scores use the benchmark’s 0–100 scale; Δ denotes HoH@3 minus Vanilla.

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Action						
Platformer	Momentum Lab	38.28	34.49	45.03	48.80	+10.52
	Ivory Beats	36.95	39.64	39.89	42.23	+5.29
	Thunder Valkyrie	60.52	50.14	61.74	64.71	+4.20
Shooter	Void Patrol	56.14	67.37	66.21	76.66	+20.52
	Wave Commander	69.66	64.54	72.85	75.52	+5.86

Table 13 (continued)

Family	Task	Vanilla	HoH@1	HoH@2	HoH@3	Δ
Roguelike	Hotline Heist	41.46	47.81	44.65	43.31	+1.84
	Dungeon Shop	25.08	40.12	47.16	46.11	+21.02
	Breach Tactics	33.70	43.79	53.61	59.56	+25.86
	Void Harvest	48.96	67.37	61.15	67.28	+18.32
Timing						
Racing	Drift Circuit	36.91	59.79	55.68	55.24	+18.33
	Rocket Trials	35.50	38.21	37.33	42.36	+6.86
	Trick Runner	41.79	51.66	59.92	59.50	+17.71
Rhythm	Note Highway	10.35	39.65	54.91	62.58	+52.23
	Beat Dungeon	41.75	55.41	53.18	64.83	+23.08
	Garden	41.70	63.78	56.48	67.03	+25.33
Sports	Skateboard Park	71.06	55.25	72.20	84.99	+13.92
	Boxing Gym	40.44	55.00	57.71	60.53	+20.09
	Archery Quest	24.25	51.31	61.23	61.85	+37.60
Strategy						
Strategy	Tower Defense	43.51	57.22	46.99	55.98	+12.47
	Chess Variant	24.75	31.46	27.15	30.88	+6.13
	Spell Tactics	46.51	37.10	47.10	53.93	+7.42
Card Game	Spire Descent	26.61	18.09	10.96	10.96	-15.65
	Poker Roguelike	3.19	30.31	50.81	49.78	+46.59
	Autobattler	55.44	46.45	50.42	53.37	-2.07
Puzzle	Sokoban Dungeon	41.98	59.80	61.21	57.66	+15.68
	Circuit Wizard	15.50	31.61	46.89	53.51	+38.01
	Pipe Crisis	53.26	35.39	39.39	37.65	-15.61
Simulation						
Tycoon	Space Colony	62.72	51.54	55.76	58.33	-4.39
	Pirate Port	35.07	73.08	71.44	65.70	+30.63
	Wildhaven	58.08	57.75	61.88	65.75	+7.66
Idle	Ant Empire	61.62	54.00	82.34	82.56	+20.94
	Factory Planet	44.02	54.35	77.25	80.35	+36.33
	Dungeon Guild	73.24	63.13	70.12	73.84	+0.60
Simulation	Kitchen Rush	16.80	42.60	49.42	62.56	+45.76
	Air Control	28.61	55.82	63.76	45.69	+17.08
	Border Check	53.54	33.74	36.83	43.44	-10.10
Adventure						
Horror	Floor 13	57.29	35.94	59.74	66.60	+9.31
	Dollhouse	35.53	55.12	67.94	71.90	+36.37
	Lighthouse	60.00	76.82	73.41	74.72	+14.72
Open World	Sky Islands	19.19	52.99	49.33	59.73	+40.54
	Airship Trader	44.42	52.55	56.95	60.52	+16.10
	Bounty	43.78	40.19	45.38	58.44	+14.66
Visual Novel	Detective Noir	40.87	46.41	66.07	58.16	+17.29
	Arcane Academy	51.85	42.98	45.88	65.71	+13.86
	Time Paradox	45.44	45.99	61.56	64.22	+18.78
Mean		42.16	49.06	55.04	58.78	+16.62

C.2 FrontierSWE Per-Task Rewards

Figure 2 reports category means under the official FrontierSWE taxonomy. The unequal task counts are shown explicitly on the horizontal axis.

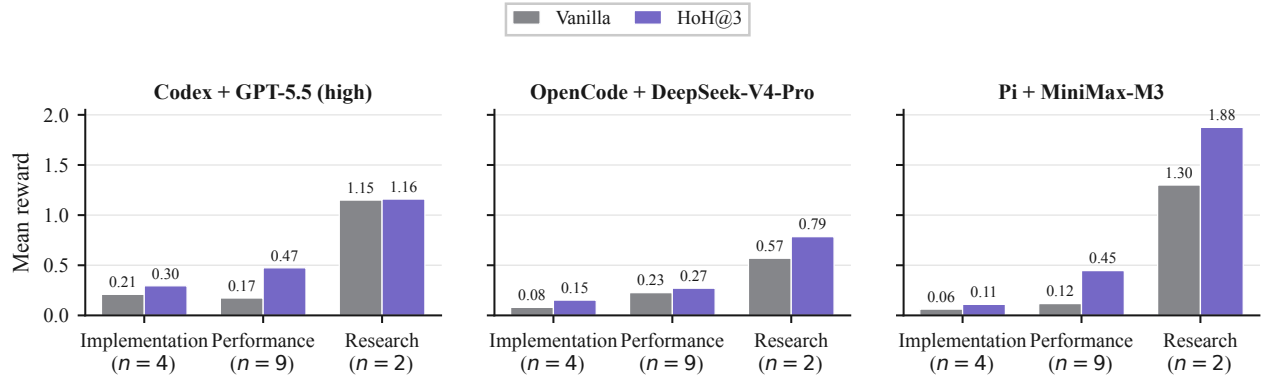


Figure 2 | FrontierSWE mean rewards by official category and harness-model configuration.

Tables 14–16 report every FrontierSWE task and condition.

Table 14 | Complete FrontierSWE task rewards for Codex + GPT-5.5 (high).

Task	Vanilla	HoH@1	HoH@2	HoH@3
Implementation				
Dart Style Haskell	0.00	0.04	0.19	0.22
Git to Zig	0.18	0.18	0.18	0.18
Lua Native Compiler	0.52	0.53	0.64	0.64
PostgreSQL–SQLite Wire Adapter	0.14	0.14	0.14	0.14
Performance				
Cranelfit Codegen Optimization	0.00	0.00	0.00	0.00
Dependent Type Checker	0.00	0.00	0.00	0.00
FFmpeg Swscale Rewrite	0.00	0.00	0.00	0.00
Granite Mamba2 Inference Optimization	0.20	1.04	1.04	0.98
Inference System Optimization	0.00	0.00	0.00	0.00
Libexpat to x86 Assembly	0.20	0.18	0.44	0.45
Notebook Compression	0.00	0.68	0.69	0.69
Pyright Type-Checking Optimization	1.16	1.01	1.16	1.19
Revideo Performance Optimization	0.00	0.85	0.94	0.96
Research				
Optimizer Design	1.40	1.52	1.17	1.42
PCQM4Mv2 Autoresearch	0.90	0.00	0.00	0.90
Mean	0.31	0.41	0.44	0.52

Table 15 | Complete FrontierSWE task rewards for OpenCode + DeepSeek-V4-Pro.

Task	Vanilla	HoH@1	HoH@2	HoH@3
Implementation				
Dart Style Haskell	0.02	0.04	0.05	0.04
Git to Zig	0.13	0.17	0.17	0.17
Lua Native Compiler	0.02	0.02	0.03	0.25
PostgreSQL–SQLite Wire Adapter	0.15	0.14	0.15	0.15
Performance				
Cranelfit Codegen Optimization	0.00	0.00	0.00	0.00
Dependent Type Checker	0.00	0.00	0.00	0.00
FFmpeg Swscale Rewrite	0.00	0.00	0.00	0.00

Table 15 (continued)

Task	Vanilla	HoH@1	HoH@2	HoH@3
Granite Mamba2 Inference Optimization	0.20	0.19	0.21	0.20
Inference System Optimization	0.00	0.00	0.00	0.00
Libexpat to x86 Assembly	0.00	0.00	0.00	0.00
Notebook Compression	0.00	0.00	0.00	0.00
Pyright Type-Checking Optimization	1.04	1.17	1.30	1.29
Revideo Performance Optimization	0.80	0.75	0.92	0.95
Research				
Optimizer Design	1.14	1.55	1.55	1.57
PCQM4Mv2 Autoresearch	0.00	0.00	0.00	0.00
Mean	0.23	0.27	0.29	0.31

Table 16 | Complete FrontierSWE task rewards for Pi + MiniMax-M3.

Task	Vanilla	HoH@1	HoH@2	HoH@3
Implementation				
Dart Style Haskell	0.12	0.04	0.06	0.07
Git to Zig	0.00	0.19	0.19	0.19
Lua Native Compiler	0.00	0.03	0.03	0.03
PostgreSQL–SQLite Wire Adapter	0.13	0.15	0.15	0.15
Performance				
Cranelfit Codegen Optimization	0.00	0.00	0.00	0.00
Dependent Type Checker	0.00	0.00	0.00	0.00
FFmpeg Swscale Rewrite	0.00	0.00	0.00	0.00
Granite Mamba2 Inference Optimization	1.06	1.97	1.98	2.18
Inference System Optimization	0.00	0.00	0.00	0.00
Libexpat to x86 Assembly	0.00	0.00	0.00	0.00
Notebook Compression	0.00	0.67	0.67	0.67
Pyright Type-Checking Optimization	0.00	1.18	1.18	1.17
Revideo Performance Optimization	0.00	0.00	0.00	0.00
Research				
Optimizer Design	2.60	2.90	2.90	2.87
PCQM4Mv2 Autoresearch	0.00	0.88	0.88	0.88
Mean	0.26	0.53	0.54	0.55

C.3 Budget-Controlled Comparison

The pass-controlled experiment uses Codex with GPT-5.5 (high) on the same 45 GameCraft-Bench tasks under the protocol in Section B.5. HoH uses three complete planning–coding–testing iterations. Figure 3 shows how artifact quality and cumulative token use change over the three development passes.

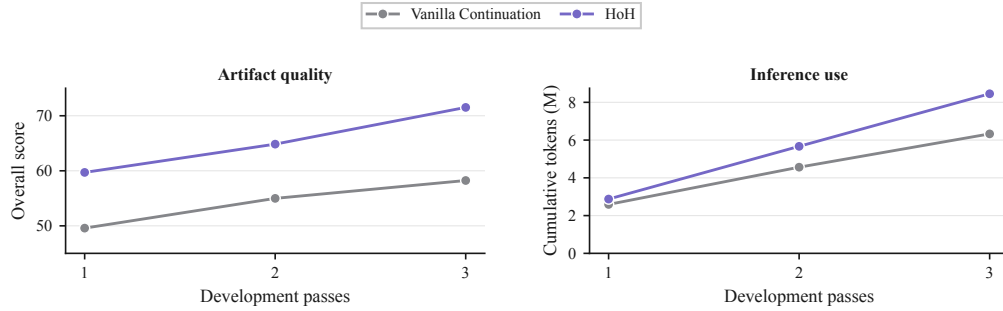


Figure 3 | Score and cumulative token trajectories in the budget-controlled GameCraft-Bench comparison using Codex with GPT-5.5 (high). HoH includes planning, coding, and testing at each pass.

Tables 17 and 18 report the task-level scores and cumulative coding-harness tokens, respectively. Averaged over the 45 tasks, Vanilla, three-pass Vanilla Continuation, and HoH obtain scores of 49.58, 58.24, and 71.52 using 2.59M, 6.33M, and 8.41M tokens per task, respectively. By Eq. 10, the three-pass conditions gain 2.32 and 3.77 score points per additional million tokens for Vanilla Continuation and HoH, respectively.

Table 17 | Task-level scores for the budget-controlled comparison on GameCraft-Bench using Codex + GPT-5.5 (high).

Task	Vanilla	Vanilla Cont.@2	Vanilla Cont.@3	HoH@3
Action				
Ivory Beats	46.81	44.58	44.84	85.42
Momentum Lab	34.05	44.51	41.15	70.61
Thunder Valkyrie	53.51	53.44	66.41	73.67
Hotline Heist	43.37	41.70	44.21	62.81
Void Patrol	58.59	74.05	72.66	87.83
Wave Commander	66.97	69.79	63.52	75.25
Void Harvest	41.05	41.06	40.60	57.43
Breach Tactics	53.44	54.04	55.93	61.09
Dungeon Shop	40.89	41.59	41.09	65.09
Timing				
Drift Circuit	43.58	41.72	63.77	70.08
Rocket Trials	45.19	43.81	49.82	70.31
Trick Runner	51.50	48.38	53.64	64.78
Beat Dungeon	47.94	46.70	50.41	60.81
Garden	52.75	56.44	58.62	70.69
Note Highway	39.44	40.84	44.28	64.68
Archery Quest	58.09	66.64	72.67	76.69
Boxing Gym	43.07	52.75	56.44	73.14
Skateboard Park	57.64	68.37	67.47	81.14
Strategy				
Chess Variant	30.04	32.89	32.46	59.72
Spell Tactics	53.10	52.98	52.75	63.32
Tower Defense	58.85	69.76	71.02	76.92
Autobattler	55.06	63.80	59.77	72.22
Poker Roguelike	44.66	48.69	51.48	69.27
Spire Descent	25.85	41.60	55.81	61.92
Circuit Wizard	31.35	34.47	36.78	49.52
Pipe Crisis	41.88	39.64	45.39	72.17
Sokoban Dungeon	55.75	65.11	67.14	70.13

Table 17 (continued)

Task	Vanilla	Vanilla Cont.@2	Vanilla Cont.@3	HoH@3
Simulation				
Pirate Port	51.32	48.05	72.20	77.81
Space Colony	36.87	43.97	45.09	76.55
Wildhaven	49.42	60.30	64.68	80.39
Ant Empire	65.52	81.58	85.84	87.88
Dungeon Guild	63.92	72.76	75.41	79.50
Factory Planet	75.16	79.27	80.69	87.78
Air Control	54.24	44.85	55.18	69.73
Border Check	43.57	69.05	69.45	72.74
Kitchen Rush	42.62	58.57	54.23	73.38
Adventure				
Dollhouse	56.33	64.55	70.36	72.59
Floor 13	48.68	65.44	66.90	73.41
Lighthouse	51.95	69.16	72.70	80.90
Airship Trader	59.58	67.71	65.15	74.41
Bounty	47.00	51.46	54.09	68.27
Sky Islands	53.25	61.56	63.38	68.25
Arcane Academy	59.30	54.02	52.10	70.69
Detective Noir	47.46	45.40	51.15	65.31
Time Paradox	50.63	57.48	61.92	71.97
Mean	49.58	54.99	58.24	71.52

Table 18 | Task-level cumulative token usage (M) for the budget-controlled comparison on GameCraft-Bench using Codex + GPT-5.5 (high).

Task	Vanilla	Vanilla Cont.@2	Vanilla Cont.@3	HoH@3
Action				
Ivory Beats	1.49	2.32	3.17	5.64
Momentum Lab	2.27	5.07	8.13	9.00
Thunder Valkyrie	1.86	4.96	6.61	8.27
Hotline Heist	1.41	2.28	3.70	7.68
Void Patrol	2.42	4.21	6.39	6.55
Wave Commander	2.61	4.66	5.82	9.37
Void Harvest	2.75	4.62	6.37	8.50
Breach Tactics	2.43	4.50	8.46	7.54
Dungeon Shop	2.53	4.99	6.33	6.31
Timing				
Drift Circuit	2.20	6.18	8.22	9.16
Rocket Trials	1.92	4.55	5.90	10.01
Trick Runner	4.13	5.83	7.68	7.65
Beat Dungeon	1.85	3.20	4.31	6.84
Garden	4.57	6.08	7.59	9.65
Note Highway	2.73	5.26	9.05	6.19
Archery Quest	1.51	5.62	7.24	11.16
Boxing Gym	3.25	5.19	6.55	7.82
Skateboard Park	2.41	3.31	4.44	7.63
Strategy				
Chess Variant	1.77	2.48	5.00	9.13
Spell Tactics	2.22	3.83	5.71	7.37
Tower Defense	2.12	3.77	6.12	11.85

Table 18 (continued)

Task	Vanilla	Vanilla Cont.@2	Vanilla Cont.@3	HoH@3
Autobattler	2.74	4.74	6.16	10.83
Poker Roguelike	3.41	7.13	9.29	7.35
Spire Descent	2.31	4.16	6.11	11.51
Circuit Wizard	2.06	4.13	6.70	7.01
Pipe Crisis	1.73	3.33	4.27	5.97
Sokoban Dungeon	1.30	2.67	3.88	6.27
Simulation				
Pirate Port	2.43	3.32	5.81	6.69
Space Colony	4.79	6.72	9.04	9.27
Wildhaven	2.58	4.01	5.22	9.93
Ant Empire	3.11	4.87	6.38	11.02
Dungeon Guild	2.41	6.81	9.33	9.28
Factory Planet	3.75	6.30	7.23	9.15
Air Control	3.25	5.32	7.38	7.97
Border Check	1.52	3.34	4.92	7.30
Kitchen Rush	2.36	4.19	5.18	8.98
Adventure				
Dollhouse	1.20	2.22	3.61	8.10
Floor 13	1.68	3.25	4.90	5.81
Lighthouse	2.91	4.25	6.07	8.24
Airship Trader	1.92	3.58	4.78	7.34
Bounty	4.18	7.97	9.94	8.32
Sky Islands	3.60	4.78	5.72	15.71
Arcane Academy	3.59	5.39	7.70	6.56
Detective Noir	3.69	4.09	4.86	8.04
Time Paradox	3.68	5.91	7.53	8.23
Mean	2.59	4.56	6.33	8.41

C.4 Ablation Study

We evaluate three variants of HoH with $T = 3$ on all 45 GameCraft-Bench tasks using Codex with GPT-5.5 (high), following the interventions in Section B.6. The complete task-level scores are reported in Table 19. Aggregate token usage is reported separately in Table 20. The mean score decreases from 71.52 for full HoH to 63.39 without plan update, 65.23 without evidence feedback, and 63.67 without artifact warm-start. Figure 4 places the score changes beside their cumulative token use.

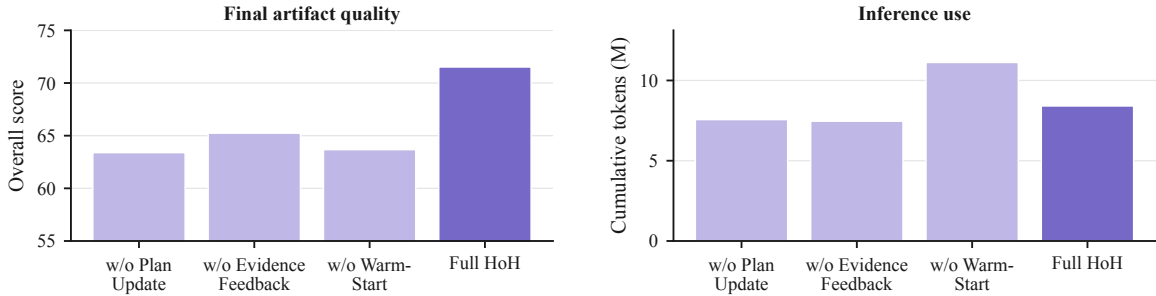


Figure 4 | Final score and cumulative token use for full HoH and the three cross-iteration ablations on GameCraft-Bench.

Table 19 | Task-level GameCraft-Bench ablation scores using Codex + GPT-5.5 (high), with $T = 3$. All values use the benchmark’s 0–100 scale.

Task	Full HoH	w/o Plan Update	w/o Evidence Feedback	w/o Warm-Start
Action				
Momentum Lab	70.61	67.81	56.67	58.77
Ivory Beats	85.42	69.42	80.33	78.85
Thunder Valkyrie	73.67	69.99	72.94	72.51
Void Patrol	87.83	73.22	77.79	72.05
Wave Commander	75.25	68.74	73.28	68.31
Hotline Heist	62.81	52.40	58.24	35.21
Dungeon Shop	65.09	57.73	59.33	64.00
Breach Tactics	61.09	58.73	53.49	52.89
Void Harvest	57.43	53.96	52.46	53.04
Timing				
Drift Circuit	70.08	63.19	66.85	52.81
Rocket Trials	70.31	63.56	68.15	64.09
Trick Runner	64.78	62.55	62.45	57.59
Note Highway	64.68	63.84	61.32	64.51
Beat Dungeon	60.81	58.38	57.50	50.84
Garden	70.69	65.61	64.19	67.89
Skateboard Park	81.14	75.77	75.24	77.44
Boxing Gym	73.14	47.90	43.73	71.16
Archery Quest	76.69	66.11	72.52	65.61
Strategy				
Tower Defense	76.92	68.56	63.55	51.71
Chess Variant	59.72	57.91	47.87	51.58
Spell Tactics	63.32	50.70	58.73	58.81
Spire Descent	61.92	46.15	60.31	60.21
Poker Roguelike	69.27	64.70	64.11	58.89
Autobattler	72.22	68.39	71.42	56.35
Sokoban Dungeon	70.13	57.02	68.60	64.19
Circuit Wizard	49.52	39.38	35.48	40.11
Pipe Crisis	72.17	62.00	61.94	46.91
Simulation				
Space Colony	76.55	57.86	75.17	75.14
Pirate Port	77.81	75.55	70.51	76.69
Wildhaven	80.39	79.00	79.12	75.22
Ant Empire	87.88	78.05	82.11	80.69
Factory Planet	87.78	70.69	83.21	84.36
Dungeon Guild	79.50	67.98	73.23	76.10
Kitchen Rush	73.38	57.69	60.10	56.94
Air Control	69.73	63.84	66.51	64.92
Border Check	72.74	65.41	62.44	61.16
Adventure				
Floor 13	73.41	68.94	69.89	63.37
Dollhouse	72.59	60.84	60.38	67.71
Lighthouse	80.90	72.72	74.72	81.50
Sky Islands	68.25	62.50	63.07	59.85
Airship Trader	74.41	61.49	69.94	71.21
Bounty	68.27	67.41	68.20	64.57
Detective Noir	65.31	57.41	51.77	60.54
Arcane Academy	70.69	64.35	67.72	57.91

Table 19 (continued)

Task	Full HoH	w/o Plan Update	w/o Evidence Feedback	w/o Warm-Start
Time Paradox	71.97	67.02	68.91	70.96
Mean	71.52	63.39	65.23	63.67

Table 20 | Mean cumulative coding-harness tokens per task for the GameCraft-Bench ablation variants.

Variant	Tokens (M)
w/o Plan Update	7.56
w/o Evidence Feedback	7.46
w/o Warm-Start	11.12
Full HoH	8.41

C.5 Resource Usage

Figure 5 shows the distribution of tokens used by each invocation on the 45 GameCraft-Bench tasks. The panels retain the native provider accounting for each harness–model configuration and should therefore be compared within, rather than across, panels.

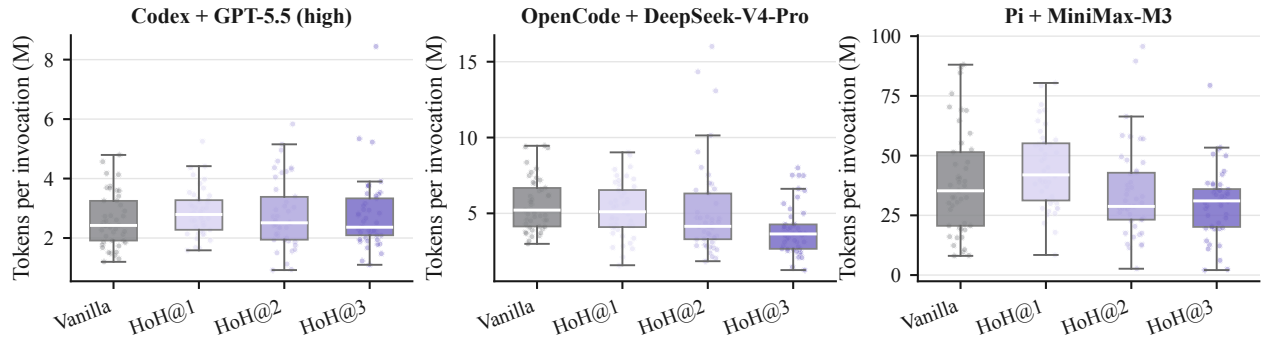
**Figure 5** | Per-invocation coding-harness token distributions on GameCraft-Bench. Points denote tasks; boxes show the median and interquartile range.

Table 21 aggregates the recorded resource use for the FrontierSWE runs.

Table 21 | Aggregate FrontierSWE resource usage.

Harness–model	Vanilla		HoH@1		HoH@2		HoH@3	
	Tokens (M)	Time (h)	Tokens (M)	Time (h)	Tokens (M)	Time (h)	Tokens (M)	Time (h)
Codex + GPT-5.5 (high)	103.43	18.65	109.26	14.32	83.19	13.21	71.71	10.28
OpenCode + DeepSeek-V4-Pro	384.84	32.00	332.61	23.58	345.66	23.32	229.77	16.16
Pi + MiniMax-M3	541.73	40.19	1117.11	49.05	988.00	21.53	717.69	37.15

D Qualitative Analysis

Figures 6–10 show one representative game from each of the 15 GameCraft-Bench families. For each family, we select the game with the highest HoH@3 Overall score under Codex with GPT-5.5 (high). Each row compares Vanilla and HoH@1–3; the values beneath each artifact report Overall, Core Mechanics (M), Content Depth (D), Functional Visuals (V), and Art and Presentation (A).

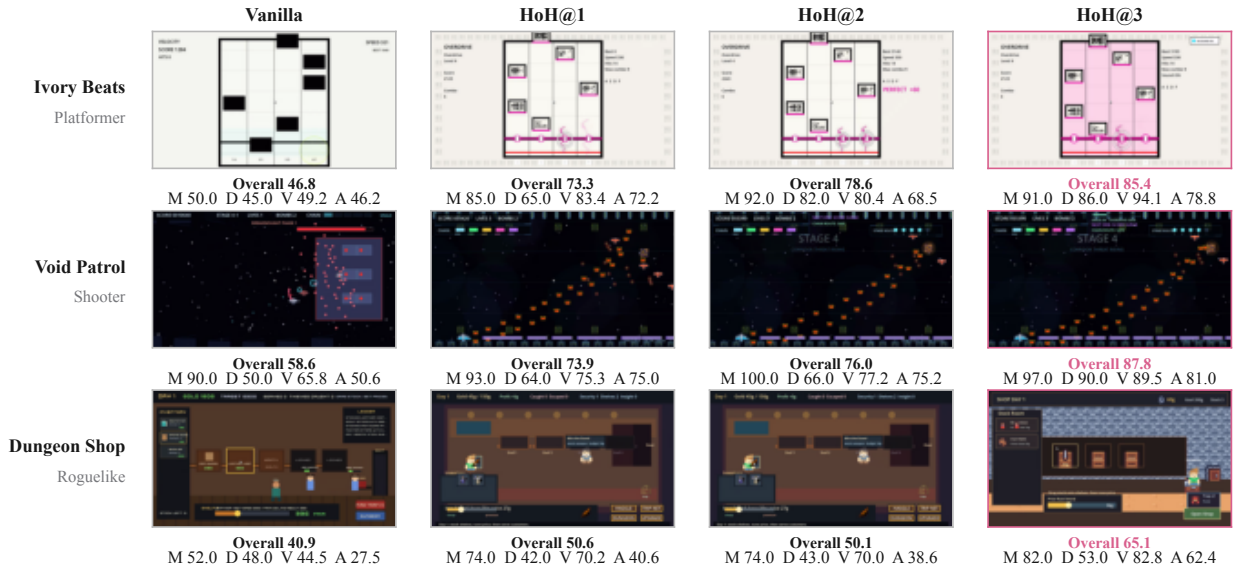


Figure 6 | Action: Platformer, Shooter, and Roguelike.

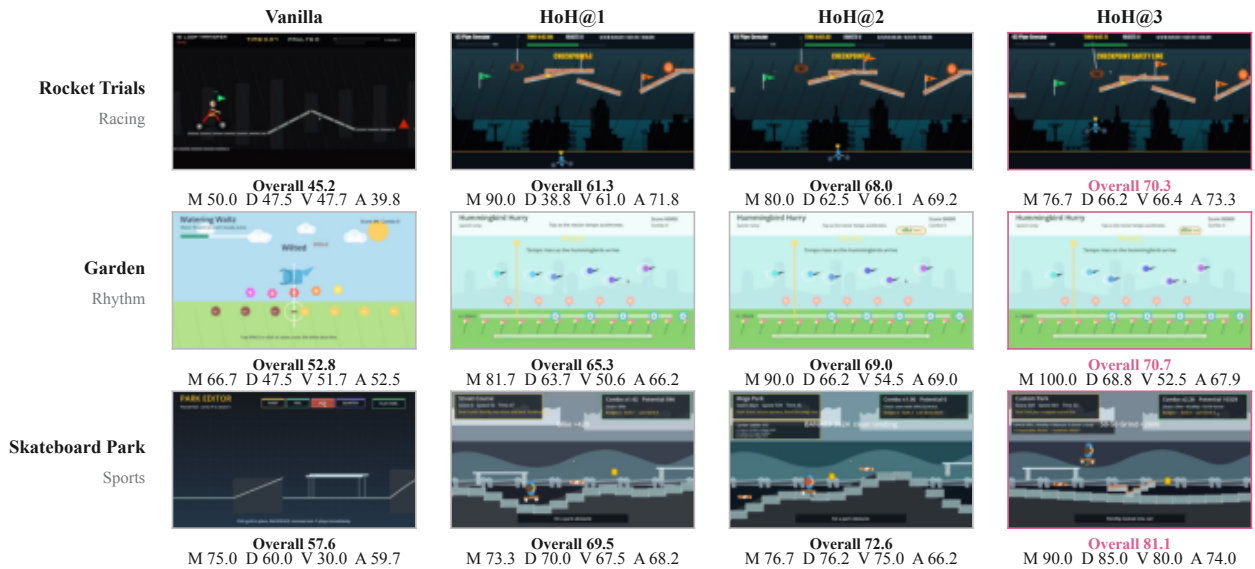


Figure 7 | Timing: Racing, Rhythm, and Sports.

	Vanilla	HoH@1	HoH@2	HoH@3
Tower Defense Strategy	 Overall 58.8 M 65.0 D 67.5 V 67.8 A 43.8	 Overall 64.0 M 67.5 D 66.2 V 80.8 A 53.0	 Overall 64.1 M 67.5 D 70.0 V 77.1 A 51.2	 Overall 76.9 M 73.8 D 75.0 V 90.2 A 74.5
Autobattler Card Game	 Overall 55.1 M 75.0 D 65.0 V 44.2 A 41.2	 Overall 72.4 M 86.7 D 76.2 V 61.7 A 67.0	 Overall 71.3 M 86.7 D 80.0 V 49.7 A 65.2	 Overall 72.2 M 91.7 D 75.0 V 61.7 A 65.6
Pipe Crisis Puzzle	 Overall 41.9 M 50.0 D 36.2 V 55.8 A 38.1	 Overall 65.1 M 86.2 D 48.8 V 81.3 A 65.3	 Overall 68.7 M 92.5 D 58.8 V 79.8 A 63.6	 Overall 72.2 M 100.0 D 60.0 V 81.7 A 68.3

Figure 8 | Strategy: Strategy, Card Game, and Puzzle.

	Vanilla	HoH@1	HoH@2	HoH@3
Wildhaven Tycoon	 Overall 49.4 M 52.0 D 44.2 V 73.3 A 43.3	 Overall 74.1 M 91.0 D 72.5 V 85.0 A 63.8	 Overall 77.8 M 89.0 D 81.7 V 91.7 A 63.1	 Overall 80.4 M 90.0 D 87.5 V 95.0 A 62.9
Ant Empire Idle	 Overall 65.5 M 93.3 D 68.8 V 35.0 A 63.4	 Overall 71.4 M 100.0 D 63.7 V 57.5 A 72.8	 Overall 69.6 M 96.7 D 67.5 V 45.0 A 70.8	 Overall 87.9 M 100.0 D 85.0 V 92.5 A 83.6
Kitchen Rush Simulation	 Overall 42.6 M 67.0 D 42.0 V 40.0 A 33.9	 Overall 48.1 M 73.0 D 49.0 V 47.8 A 36.6	 Overall 65.6 M 90.0 D 68.0 V 65.2 A 53.0	 Overall 73.4 M 100.0 D 72.0 V 72.9 A 63.5

Figure 9 | Simulation: Tycoon, Idle, and Simulation.



Figure 10 | Adventure: Horror, Open World, and Visual Novel.

Table 22 reports the exact Overall and dimension scores underlying all 60 artifacts shown above.

Table 22 | Scores for the 15 GameCraft-Bench games shown in the qualitative comparison. One game is selected per benchmark family by the highest HoH@3 Overall score under Codex with GPT-5.5 (high).

Family	Game	Condition	Overall	Mechanics	Depth	Visuals	Art
Action							
Platformer	Ivory Beats	Vanilla	46.81	50.00	45.00	49.17	46.25
		HoH@1	73.28	85.00	65.00	83.44	72.19
		HoH@2	78.55	92.00	82.00	80.42	68.54
		HoH@3	85.42	91.00	86.00	94.06	78.75
Shooter	Void Patrol	Vanilla	58.59	90.00	50.00	65.83	50.62
		HoH@1	73.90	93.00	64.00	75.31	75.00
		HoH@2	76.02	100.00	66.00	77.25	75.25
		HoH@3	87.83	97.00	90.00	89.50	81.00
Roguelike	Dungeon Shop	Vanilla	40.89	52.00	48.00	44.46	27.50
		HoH@1	50.55	74.00	42.00	70.21	40.62
		HoH@2	50.15	74.00	43.00	70.00	38.57
		HoH@3	65.09	82.00	53.00	82.75	62.38
Timing							
Racing	Rocket Trials	Vanilla	45.19	50.00	47.50	47.67	39.75
		HoH@1	61.33	90.00	38.75	61.00	71.75
		HoH@2	68.00	80.00	62.50	66.11	69.17
		HoH@3	70.31	76.67	66.25	66.39	73.33
Rhythm	Garden	Vanilla	52.75	66.67	47.50	51.67	52.50
		HoH@1	65.33	81.67	63.75	50.56	66.25
		HoH@2	69.01	90.00	66.25	54.50	69.00
		HoH@3	70.69	100.00	68.75	52.50	67.88
Sports	Skateboard Park	Vanilla	57.64	75.00	60.00	30.00	59.69
		HoH@1	69.51	73.33	70.00	67.50	68.25
		HoH@2	72.62	76.67	76.25	75.00	66.25
		HoH@3	81.14	90.00	85.00	80.00	73.96

Table 22 (continued)

Family	Game	Condition	Overall	Mechanics	Depth	Visuals	Art
Strategy							
Strategy	Tower Defense	Vanilla	58.85	65.00	67.50	67.75	43.75
		HoH@1	63.97	67.50	66.25	80.75	53.00
		HoH@2	64.12	67.50	70.00	77.08	51.25
		HoH@3	76.92	73.75	75.00	90.25	74.50
Card Game	Autobattler	Vanilla	55.06	75.00	65.00	44.17	41.25
		HoH@1	72.39	86.67	76.25	61.67	67.00
		HoH@2	71.28	86.67	80.00	49.72	65.21
		HoH@3	72.22	91.67	75.00	61.67	65.62
Puzzle	Pipe Crisis	Vanilla	41.88	50.00	36.25	55.83	38.06
		HoH@1	65.07	86.25	48.75	81.33	65.33
		HoH@2	68.65	92.50	58.75	79.76	63.57
		HoH@3	72.17	100.00	60.00	81.67	68.33
Simulation							
Tycoon	Wildhaven	Vanilla	49.42	52.00	44.17	73.33	43.33
		HoH@1	74.09	91.00	72.50	85.00	63.75
		HoH@2	77.78	89.00	81.67	91.67	63.12
		HoH@3	80.39	90.00	87.50	95.00	62.89
Idle	Ant Empire	Vanilla	65.52	93.33	68.75	35.00	63.44
		HoH@1	71.42	100.00	63.75	57.50	72.81
		HoH@2	69.64	96.67	67.50	45.00	70.75
		HoH@3	87.88	100.00	85.00	92.50	83.57
Simulation	Kitchen Rush	Vanilla	42.62	67.00	42.00	40.00	33.93
		HoH@1	48.07	73.00	49.00	47.81	36.56
		HoH@2	65.64	90.00	68.00	65.25	53.00
		HoH@3	73.38	100.00	72.00	72.92	63.54
Adventure							
Horror	Lighthouse	Vanilla	51.95	61.67	65.00	35.00	42.00
		HoH@1	63.89	65.00	63.75	50.00	69.50
		HoH@2	71.47	78.33	75.00	57.50	71.00
		HoH@3	80.90	86.67	86.25	70.00	77.75
Open World	Airship Trader	Vanilla	59.58	73.33	48.75	45.00	70.75
		HoH@1	73.60	90.00	67.50	95.00	63.50
		HoH@2	74.40	90.00	70.00	90.00	65.42
		HoH@3	74.41	95.00	70.00	87.50	64.38
Visual Novel	Time Paradox	Vanilla	50.63	77.00	60.00	40.31	34.38
		HoH@1	63.28	77.00	64.00	60.62	57.81
		HoH@2	71.15	79.00	70.00	77.92	66.04
		HoH@3	71.97	90.00	66.00	86.61	63.93